Module 1

Introduction

Choosing a language is like choosing a bar. - Josh Bloch

What problem does a programming language solve?

Early computers

How does a mechanical calculator work?

A late example looks like this.

It has important features:

- Data stored in registers
- Can do simple operations (add, subtract, multiply, divide)

Early computers

What is the function of the bottom register (counter)?

Entering ‘11’ in the keypad and pressing ‘+’ 11 times results in 121.

That's multiplication!

What is a better way?

Better multiplication

```
                                        v   <-  cursor
0   0   0   0   0   0   0   0   0   0   1   1   <-  Accumulator

0   0   0   0   0   0   0   0   0   0   1   1   <-  Keypad
```

Shift accumulator to the right.

Better multiplication

```
                                        v       <-  cursor
    0   0   0   0   0   0   0   0   0   0   1   1   <-  Accumulator


0   0   0   0   0   0   0   0   0   0   1   1       <-  Keypad
```

Press ‘+’ to add again:

```
    0   0   0   0   0   0   0   0   0   1   2   1   <-  Accumulator
```

Think Pair Share…

Let’s do some more complex stuff. How about multiplication?

Now… Division?

Pair and reflect:

1. Is this device a **computer**?
2. Why or why not?

Back to computers…

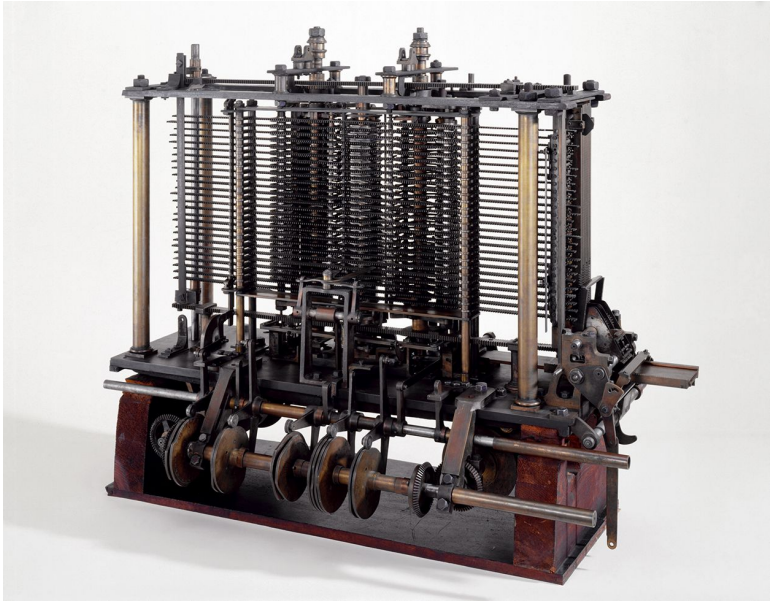
Can this device be programmed?

What needs to be done in order to add a new function, such as square root?

Brief Computer History

Charles Babbage's "analytical engine" is widely considered to be the first general purpose computer. (~1830)

Ada Lovelace, a documentation translator for this machine is considered the author of the world's first computer program.



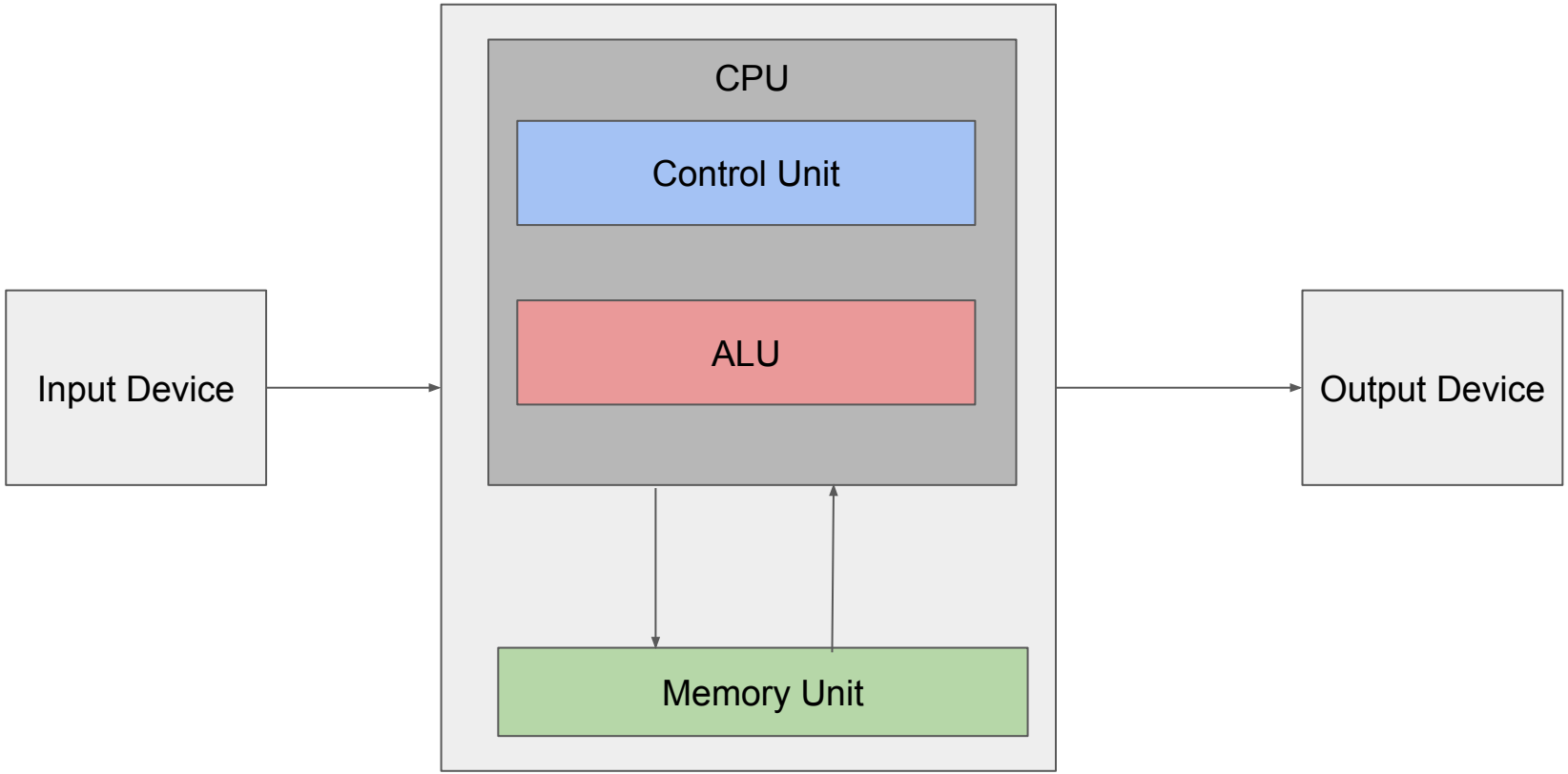
Picture credit.

Brief computer history

John von Neumann was instrumental to developing the computer in the 1930's, alongside Alonzo Church, Alan Turing and others.

Although named for him, the von Neumann architecture was the work of many people.

von Neumann Architecture



von Neumann Architecture

Computer programs are data in the computer's memory!

Bonus:

Storing the data is another engineering adventure entirely…

Early computer memory:

- Core-rope memory
- Delay-line memory

Assembly/Machine Language

Early computers used machine language to specify instructions. As you might expect, this is not easy for humans to read.

- x86_64 sum program.

In the 1940's and 1950's, Grace Hopper advocated for computer languages independent of computer code, and built a linker by 1952.

The first programming languages and compilers were invented before 1960.

What problem does a PL solve (revisited)

1. Abstract the programmer from the machine underneath (expression).
2. Assist a programmer in thinking in specific ways about problems.

The technical goal of every computer language is translate the input language into something that runs in the computer language. Remember this!

Why so many programming languages?

- Intellectual evolution: technology has progressed!
- Commercial advantage and proprietary software
- Some programming languages are oriented to special purposes (Julia, Ada, R, SQL and more).
- Humans have preferences (C is better than Java and other unsolvable arguments)
- Some are easy to learn
- Some are easy to write a compiler for
- Some are better for error prevention
- Some have huge libraries of past implementation

What makes a PL successful?

- Some are easy to learn (BASIC, Pascal, LOGO, Scheme)
- Some are easy to express things, easy use once fluent, "powerful” (C, Common Lisp, APL, Algol-68, Perl)
- Some are easy to implement (BASIC, Forth)
- Some are possible to compile to very good (fast/small) code (Fortran)
- Some have the backing of a powerful overlords (COBOL, PL/1, Ada, Visual Basic)
- Some enjoyed wide dissemination at minimal cost (Pascal, Turing, Java)

Why Study PLs?

Understanding how they work allows you to make tradeoffs between implementation costs and alternative ways of doing things.

You are nearly 100% likely to experience a conversation in industry where someone suggests using a new programming language for a project.

- You want to have more to say than “I like this one more.”

Sometimes you are stuck in a programming language that you don’t prefer.

Studying PL’s helps you understand how to work around that language's limitations to do what you want.

Types of Programming Languages

Two major classes

- Imperative
- Declarative

These are not strict boundaries!

Imperative Languages

A programmer tells the computer **how** to solve a problem.

You already know a few of these.

Subtypes:

1. von Neumann: C, Basic, Fortran
2. Object Oriented: Smalltalk, C#, Java
3. Scripting: Perl, Bash, Awk, Python, Javascript

Key features: syntax control flow, names, statements

Declarative Languages

A programmer tells the computer **what** problem to solve.

You probably don't know a declarative PL, but have used at least one.

You'll learn to program in two strictly declarative languages in CS 354.

Subtypes:

- Functional: Lisp, Scheme, Haskell, OCAML, Scala, Erlang
- Logic/constraint based: Prolog, SQL, Make, Answer Set Programming

Key features: expressions, recursion, backtracking, unusual stuff: currying, monads, continuations, procedural abstraction

Declarative Languages Continued

These languages work by describing all acceptable **solutions**. A compiler is then free to choose any acceptable way to meet the constraints.

Logic languages exemplify this:

$\texttt{sqrt}(x^2) = x.$

This is a program for square root of a number.

Functional PLs are based on recursive-function definition and invocation, without side-effects.

Types of PL Translators

Translator: a Systems Program that converts from one computer language to another.

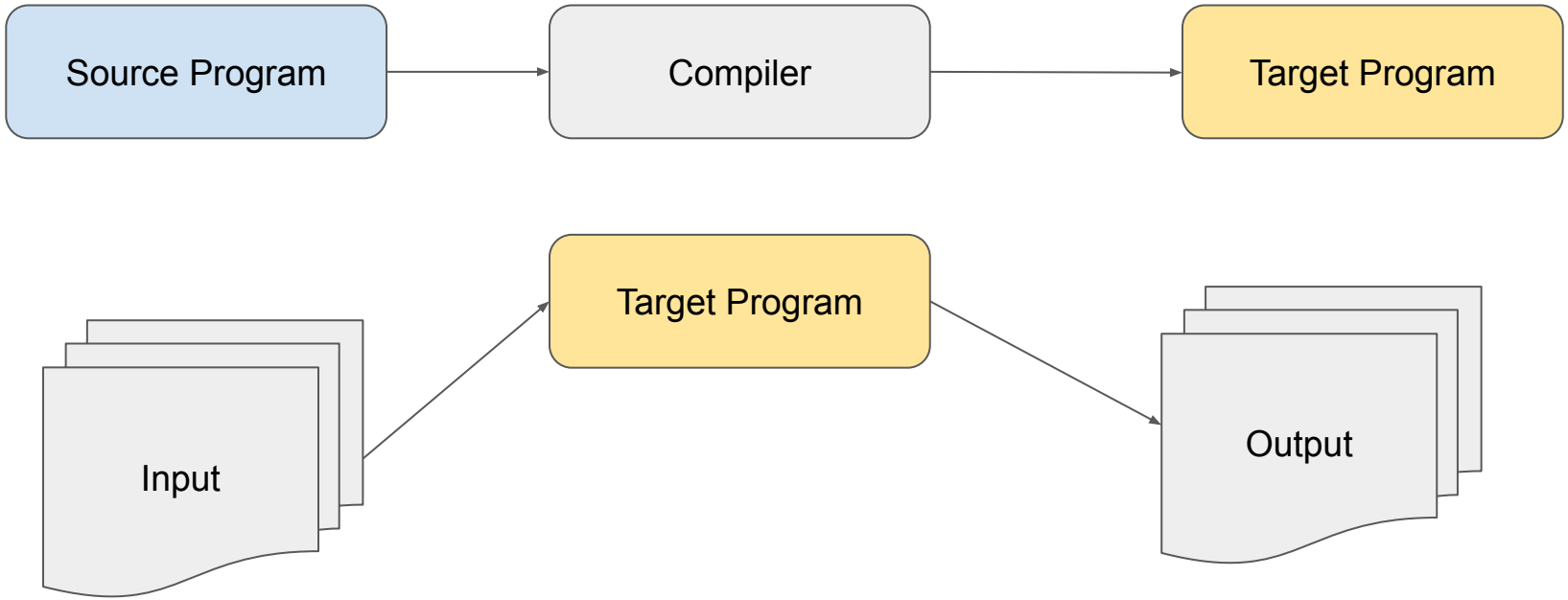
Source program -> Target Program

Major approaches:

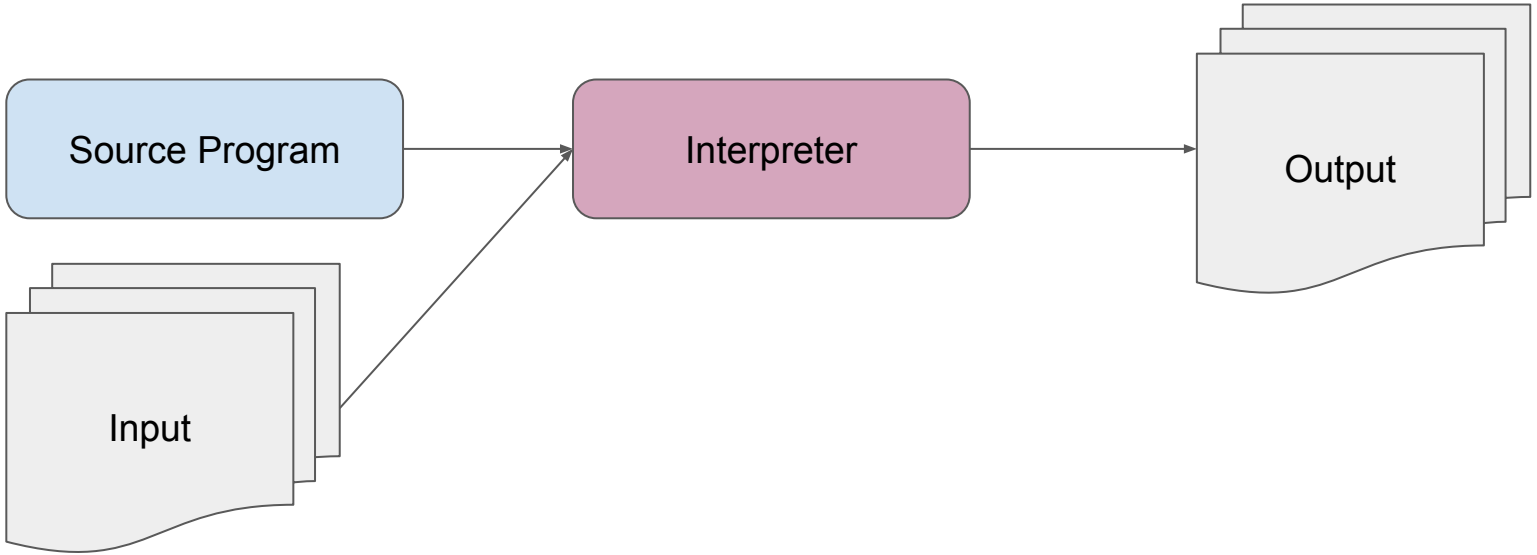
1. Assembler
2. Pure compilation
3. Interpretation
4. Hybrid

In each approach, the Compiler, Interpreter or Virtual Machine are themselves computer programs already in runnable form.

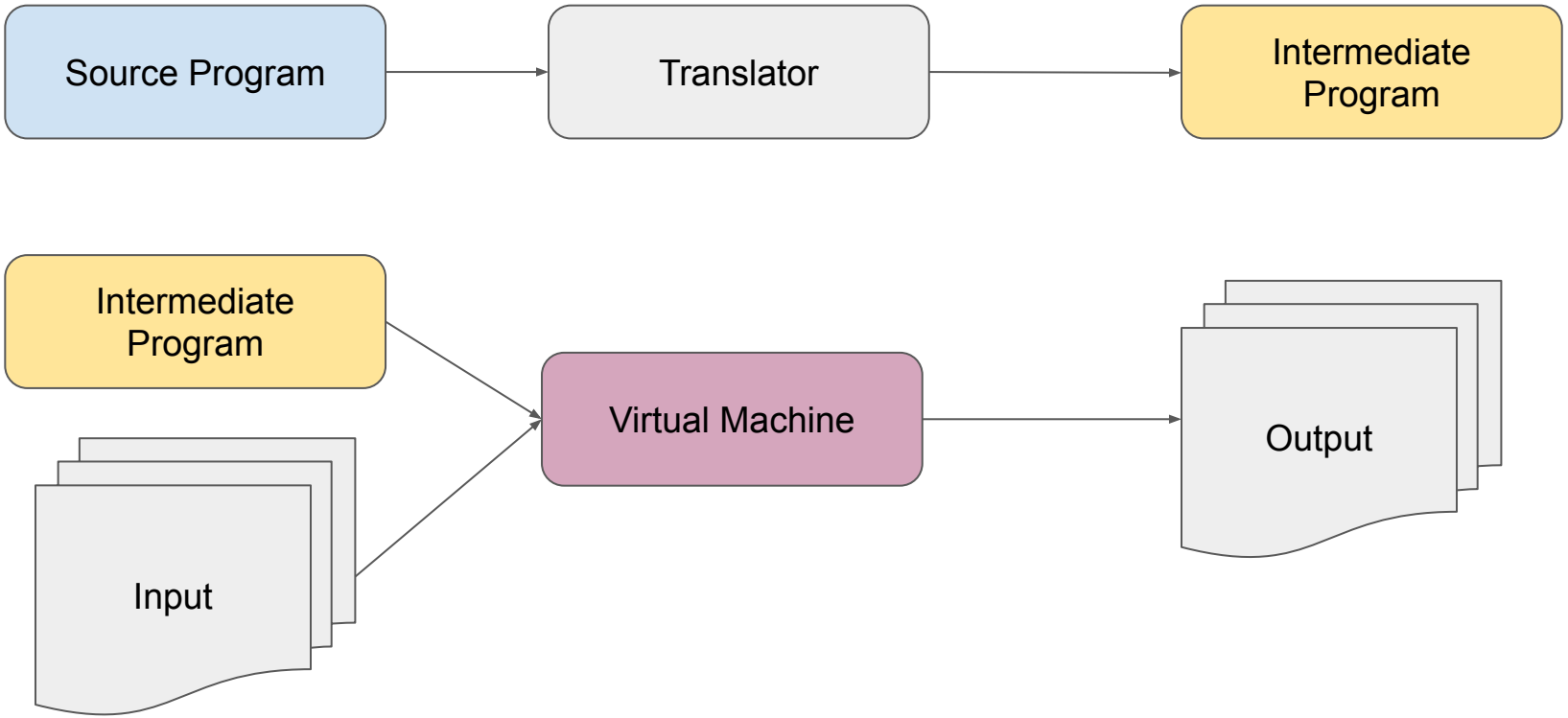
Pure Compilation



Interpretation



Hybrid


Source Program
Translator
Intermediate Program
Intermediate Program
Input
Virtual Machine
Output

Just in Time (JIT) compilation

Another common buzzword in this world is **Just In Time** Compilation.

This is not actually a distinct style, but an opportunistic optimization technique.

https://www.youtube.com/watch?v=d7KHAVaX_Rs

Types of Translators continued

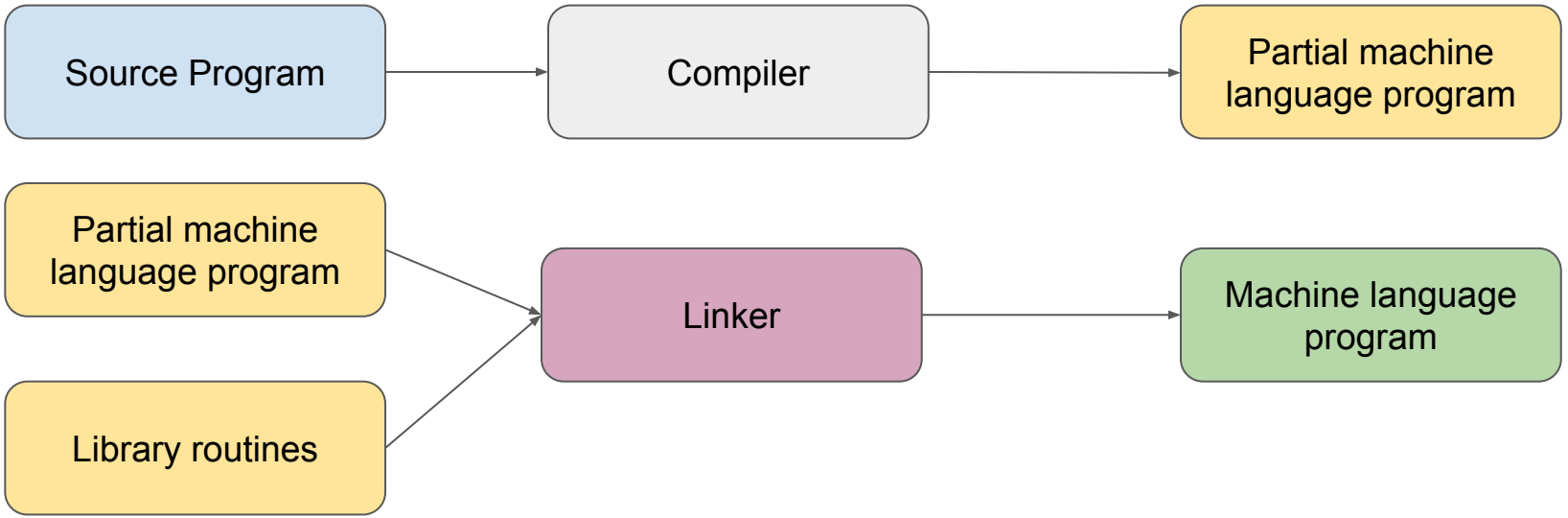
These are broad categories, and again, there are often no strict boundaries.

For example, C is strictly compiled, right? It actually needs interpreted features:

Shown here with scanf/printf.

Other types of translators

Libraries and Linking

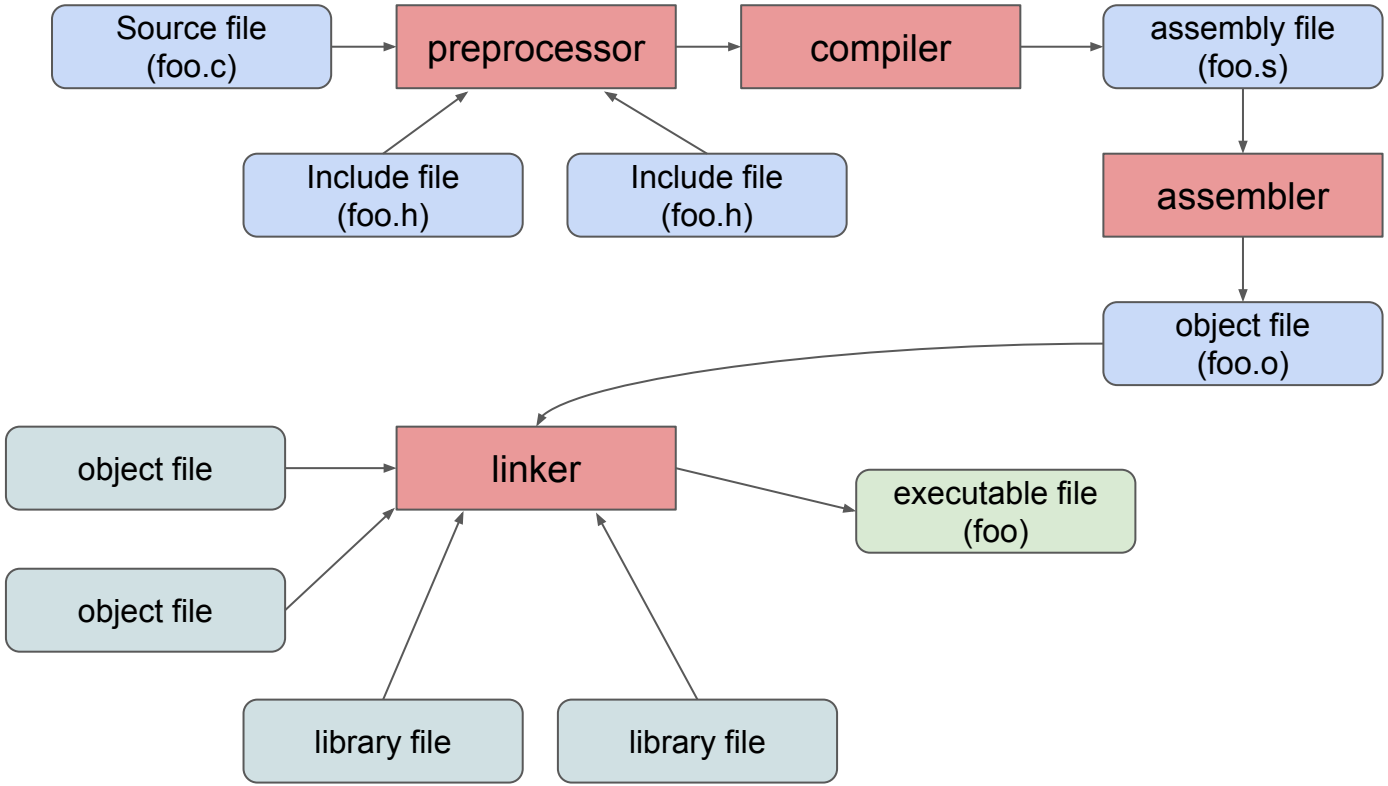


Other types of translators

Other types:

- Post compilation assembly language
- Preprocessing (C macros for example)
- Source to source translation (C++ initially, Kotlin, Typescript)
- Dynamic and Just in Type Compilation (C# generates CIL, which is translated to machine code just before runtime)

GCC Toolchain (example)



Compilers/Interpreters

The C compiler is written in C!

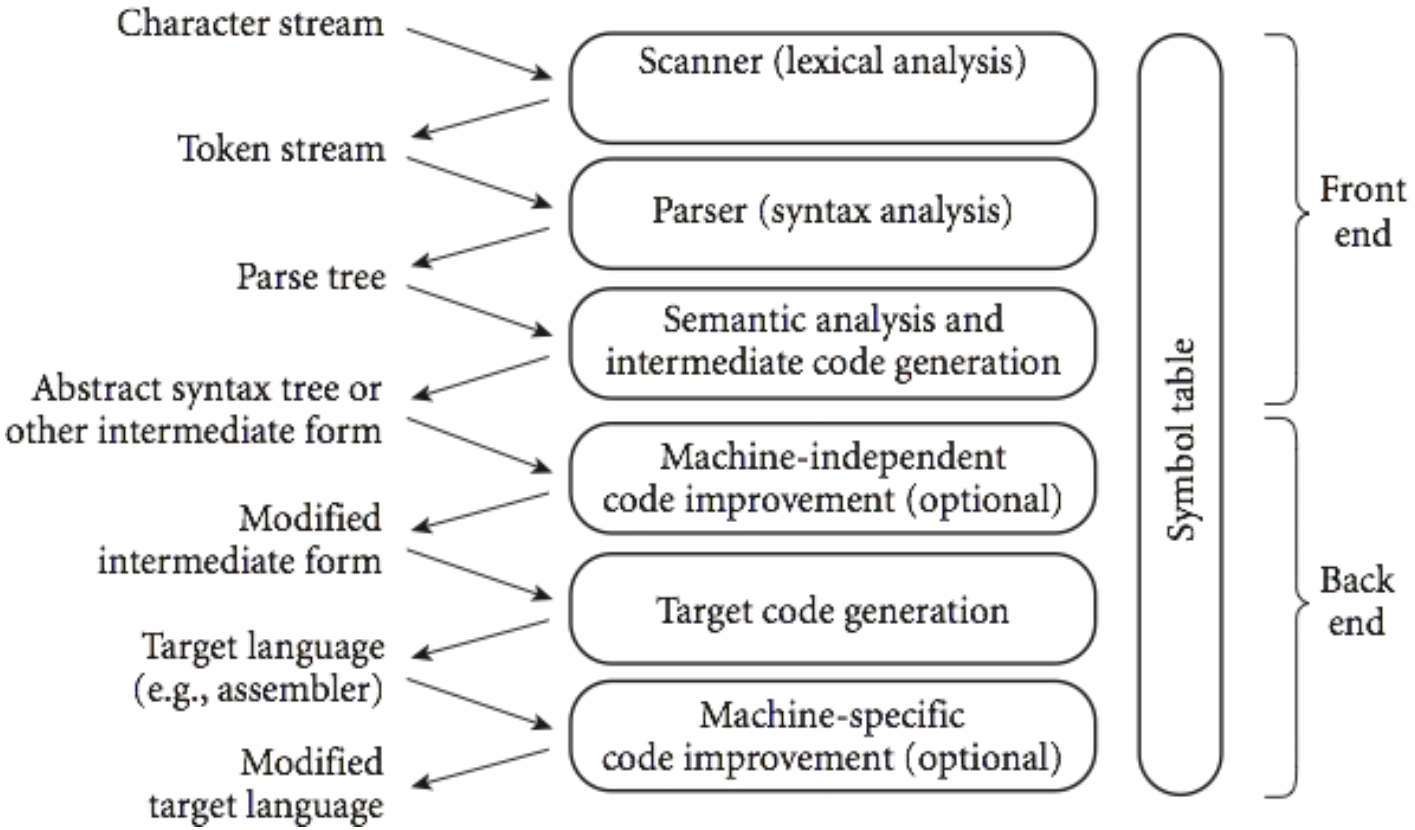
This is traditional for programming languages, but leads to a chicken and egg problem.

The process of getting this to work is called “bootstrapping”

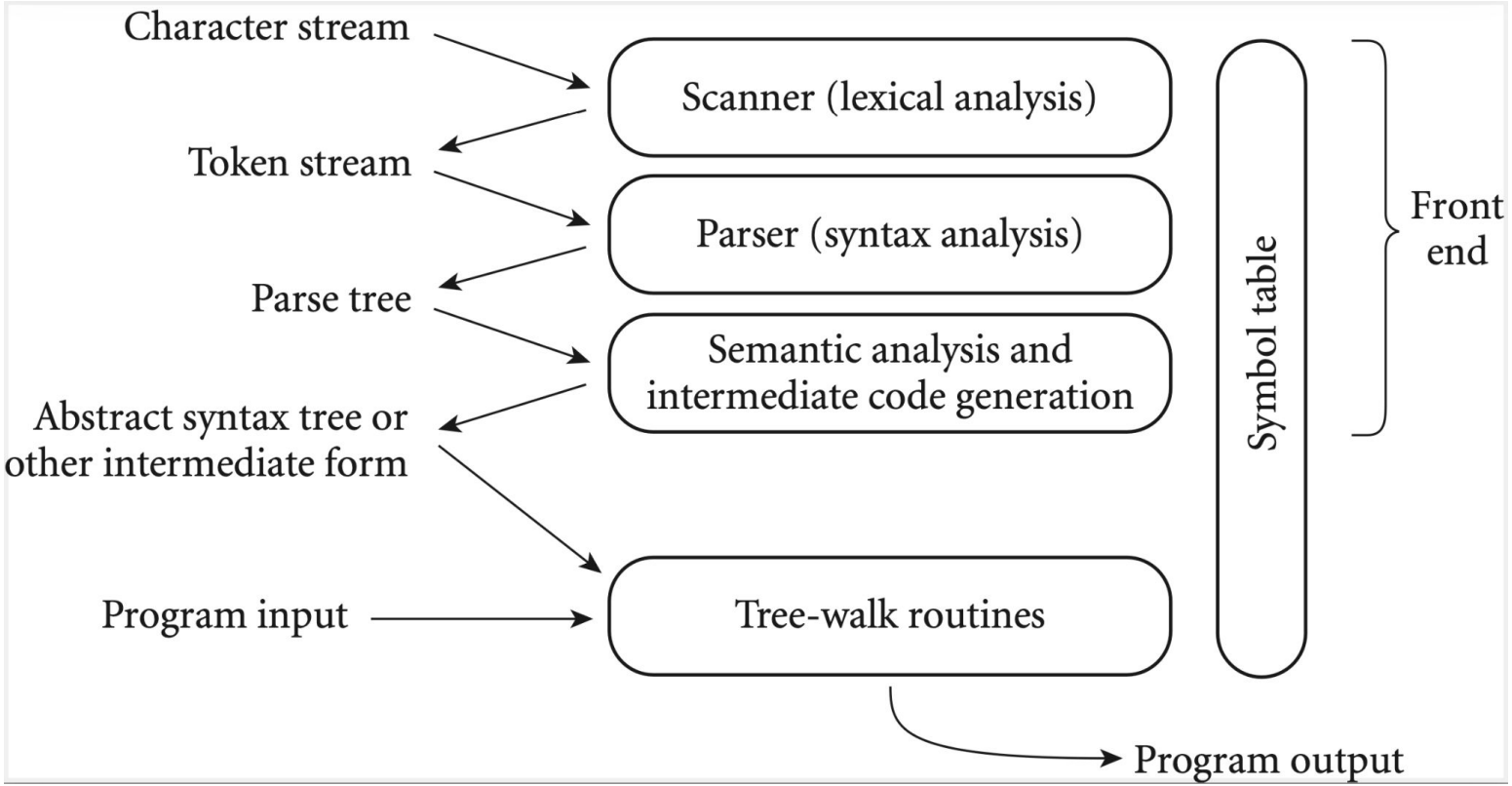
What does a compiler do?

What is an interpreter?

Phases of Compilation



Phases of Interpretation



Phase I: Scanning

- Divides the program into "tokens"
- Tokens are the smallest meaningful units; this saves time, since character-by-character processing is slow
- we can tune the scanner better if its job is simple; it also saves complexity (lots of it) for later stages
- A **parser** can be designed to take characters instead of tokens as input, but it isn't pretty
- Scanning is recognition of a regular language via DFA (CS361)

Phase II: Parsing

- Parsing is recognition of a context-free language via Push Down Automata (PDA)
- Parsing discovers the "context free" structure of the program

- Informally, it finds the structure you can describe with syntax diagrams
- SQLite has good examples of these

Here is an example of a context free grammar for C and for Java

Phase III: Semantic Analysis

- Semantic analysis is the discovery of **meaning** in the program
- The compiler does **static semantic analysis**. Static implies compile time
- Some things (e.g., array subscript out of bounds) can't be figured out until run time. Things like that are part of the program's **dynamic semantics**

Static vs. dynamic is recurring in CS 354

Phase IV: Intermediate Form (IF)

- IFs are often chosen for machine independence, ease of optimization, or compactness (these are somewhat contradictory)

- They often resemble machine code for some imaginary idealized machine; e.g. a stack machine, or a machine with arbitrarily many registers

- Many compilers actually move the code through more than one IF

Phase V and VI: Optimization and Code Generation

- **Optimization** takes an intermediate-code program and produces another one that does the same thing faster, or in less space
- The term is a misnomer; we just improve code
- The optimization phase is optional
- **Code generation** phase produces assembly language or (sometime) relocatable machine language or (sometime) high-level language

Phase VII: Machine Optimizations

- Certain machine-specific optimizations (use of special instructions or addressing modes, etc.) may be performed during or after target code generation

Examples: x86-64 has an advantage in performance when reading and writing 8-byte words. This means in many cases using smaller data types can be slower!

Important! Symbol Table

- All phases rely on a symbol table that keeps track of all the identifiers in the program and what the compiler knows about them

- This symbol table may be retained (in some form) for use by a debugger, even after compilation has completed (compilers normally accept some kind of flag for this)

Bootstrapping Pascal

Example: Pascal shipped the following to new users:

1. A Pascal compiler, written in Pascal, that would output **intermediate P-code**, a simple stack language that could be more easily interpreted than Pascal.
2. Same as 1, but written in P-code
3. A P-code interpreter, written in Pascal

This is confusing. Here is the key to understanding it:

For each component, there are 3 variables:

1. What the component is written in
2. What input language it accepts
3. What output language it generates

Bootstrapping Pascal: Components

1. A Pascal compiler written in Pascal, outputs P-code
2. A Pascal compiler written in P-code, outputs P-code
3. A P-code interpreter, written in Pascal

A user would translate #3 by hand to produce #4

4. A P-code interpreter, written in a local language (ASM)

A user then runs #2 on top of #4 to produce output P-code

Bootstrapping Pascal

#1 could then be modified to produce ASM instead of P-code

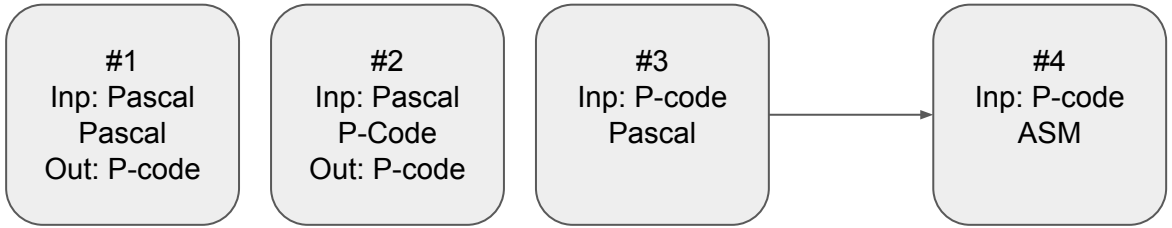
This produces #5: a Pascal Compiler, written in Pascal, that generates ASM.

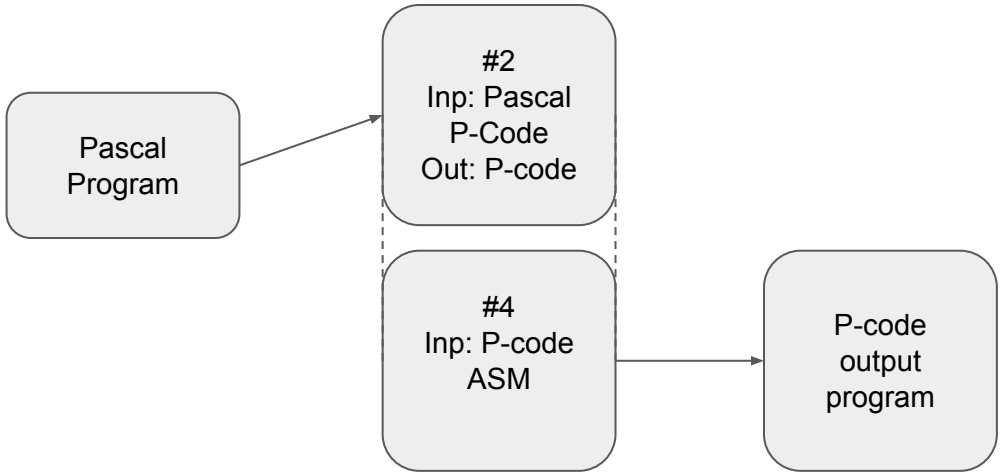
The user then runs #5 through #2 running on #4 to produce #6: an ASM version of the Pascal Compiler that inputs Pascal and outputs ASM

Bootstrapping Pascal

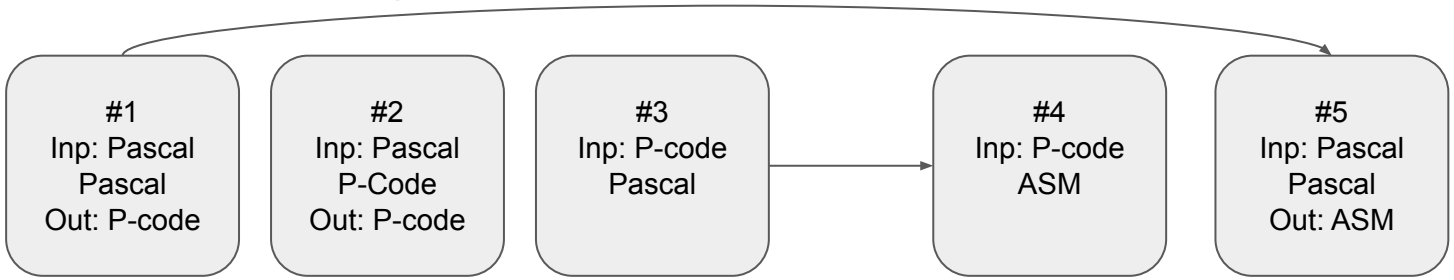
1. A Pascal compiler written in Pascal, outputs P-code
2. A Pascal compiler written in P-code, outputs P-code
3. A P-code interpreter, written in Pascal
4. A P-code interpreter, written in a local language (ASM)
5. A Pascal Compiler, written in Pascal, that outputs ASM
6. A Pascal Compiler, written in ASM, that outputs ASM

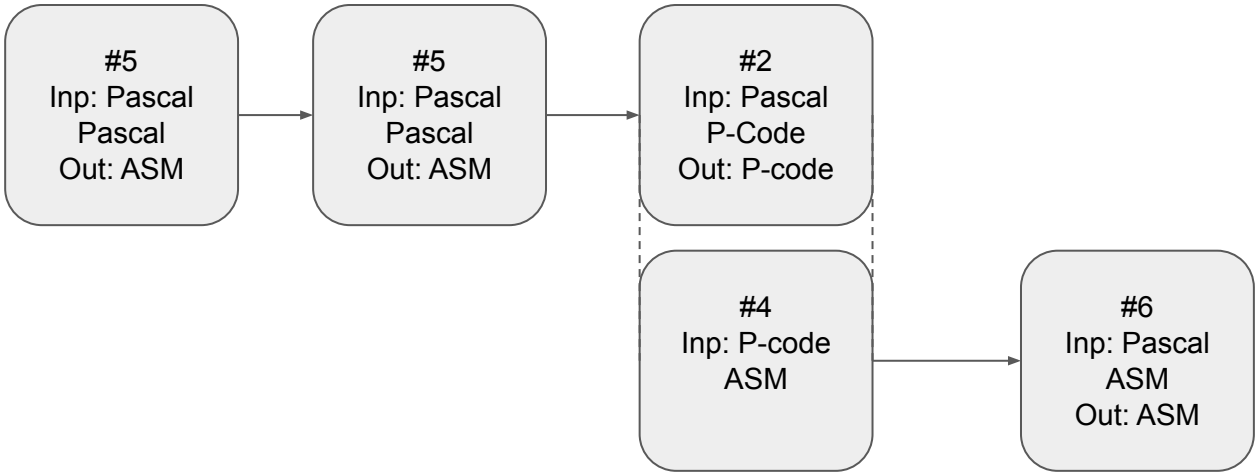
Bootstrapping Pascal





Bootstrapping Pascal





Bootstrapping *fin*

As you can see, this process is very clever. There is an analogous process in starting an operating system (booting).

Read another version of this here: https://en.wikipedia.org/wiki/Bootstrapping_(compilers)

Formula:

- Start with a limited featureset to create bootstrap compiler
- Add features to the bootstrap compiler (as in: while loops, exceptions)
- Compile new compiler with bootstrap compiler
- Repeat

For OS concept: https://en.wikipedia.org/wiki/Booting